FINAL YEAR PROJECT | DRAFT RESEARCH REPORT

PhishGuard-PK

Multilingual Scam Message and URL Risk Detection
English | Roman Urdu | Urdu

Project type: BS Computer Science final-year project prototype
Build date: 1 November 2026
Student name / institution / supervisor: to be completed by the project owner

Research integrity notice

The supplied application runs, but the currently available training labels are provisional and some cases are synthetic. This draft is not an externally validated security-system report or an advisor-approved thesis.

01 | Executive summary and objectives

PhishGuard-PK is a multilingual educational prototype for assessing user-supplied SMS, WhatsApp, email and social-media message text. It supports English, Roman Urdu and Urdu-script input. An offline URL inspector, a probabilistic text model, safety indicators, and explanatory features combine into a user-facing Safe / Suspicious / Scam verdict.

System objectives

• Accept all three languages and normalize Roman Urdu spelling and common Urdu Unicode variants.

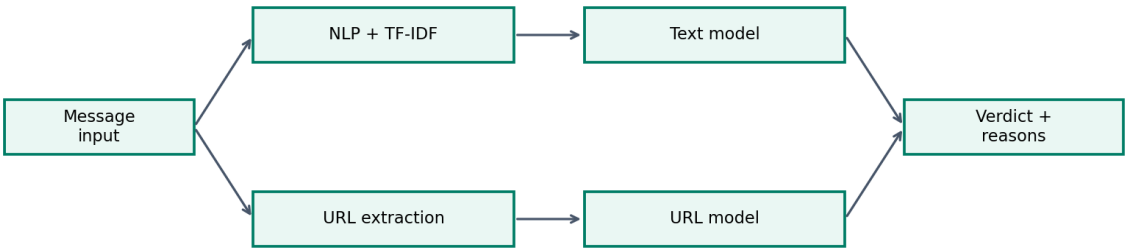
• Compare three machine-learning baselines with a grouped train/validation/test split.

• Inspect link features without opening suspected websites; support optional WHOIS registration age.

• Show a numerical risk indicator, plain-language rationale, and reproducible SHAP examples.

• Expose analysis through a simple web interface and FastAPI POST /analyze.

High-level architecture



Input text is screened by the language-aware text pipeline and a separate offline URL-analysis branch. The merged output includes a verdict, language estimate, risk indicator and reasons.

Scope boundary

The prototype accepts pasted text and URLs, not screenshot OCR, voice, video or direct access to WhatsApp accounts. It does not fetch suspicious websites. Telegram integration is optional in the supplied requirements and has not been added.

02 | Data sources and provenance

The starting audit contained 8,155 source records from earlier research, reduced to 7,353 distinct records after redaction. These are source candidates, not automatically verified Safe / Suspicious / Scam ground truth.

Stage	Rows	Interpretation
Prior audit source rows	8,155	Source-record inventory
Deduplicated source candidates	7,353	Provisional public-source labels only
Development training rows	5,750	Selected weak-label candidates plus 213 synthetic examples
Training-approved examples	0	No independent individual-label review completed

Development corpus composition

Language field	Safe	Scam	Suspicious
English	977	44	15
Roman Urdu	607	427	15
Urdu	35	44	15
Unverified Latin	2,476	1,095	0

The Unverified Latin category must not be mislabeled as English. Genuine Urdu-script and Suspicious examples are particularly underrepresented. A portion of the development corpus is source-labeled but not individually reviewed; 213 examples were written as synthetic illustrations of documented local scam formats.

Crucial labeling exclusion

Promotional messages and generic spam are NOT automatically phishing. Ambiguous source-label categories were excluded rather than silently reclassified as Suspicious/Scam. Original provenance and group identifiers are retained in the CSVs.

03 | Model methodology

Text normalization

Unicode NFKC and selected Urdu character variants are normalized. Roman Urdu spellings such as kia/kiya/kya are mapped consistently. URL, phone and email indicators are preserved as tokens. Numbers are generalized. Optional stopword removal retains important negation terms.

Feature extraction

Character-based TF-IDF n-grams of lengths 3-5 represent text. The approach is tolerant of some spelling variation and code-mixing. The trained baseline estimators are Complement Naive Bayes, calibrated Linear SVM and Random Forest.

Reproducible split and selection

Grouping by message-template family reduced simple near-duplicate leakage. Split counts: train 4088, validation 822, test 840. The intended allocation is approximately 70/15/15; actual group sizes cause small deviations. Best model was selected by validation macro F1, then evaluated on the weak-label holdout.

Why macro F1 matters

Accuracy can look high when Safe and Scam dominate. Macro F1 averages F1 equally across Safe, Suspicious and Scam, exposing weak performance on the smallest class. Validation results and test results are kept separate in this repository.

Multilingual transformer

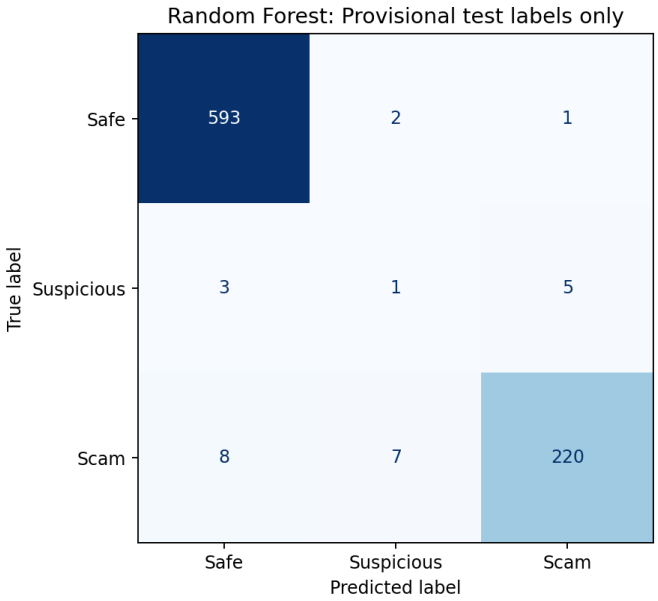
An optional reproducible XLM-RoBERTa training script, with GPU-aware Hugging Face Trainer code, is included. It was not executed in the local CPU build, and no transformer results are claimed.

04 | Experimental results - provisional labels

Important: The following figures measure agreement with weak public source labels and synthetic scenarios. They do NOT constitute verified real-world fraud-detection accuracy.

Model	Weak test accuracy	Weak test macro F1
Naive Bayes	96.67%	0.692
Linear SVM	97.86%	0.654
Random Forest	96.90%	0.683

On the validation partition, Random Forest ranked first by macro F1. Its weak-label test accuracy was 96.90%, and macro F1 was 0.683. Its Suspicious-class F1 was only 0.105 on nine test examples.



A strong-looking overall accuracy hides important minority-class errors. The language-specific weak test for Urdu had 18 examples (largely synthetic), with only 44.4% accuracy; it is not ready for security use.

05 | URL analysis and combined risk

URL features

The URL module extracts web links using a regular expression and inspects URL length, HTTPS, special characters, hyphens, digits, IP addresses, shortened links, subdomain depth, suspicious keywords, punycode and optional domain registration age.

The project never visits a supplied URL. WHOIS domain-age lookup is coded but disabled by default. Domain age is explicitly unknown when a lookup is unavailable; no date is invented.

Classifier and indicator

The URL Random Forest was trained and tested only on generated examples using safe hostnames and reserved invalid domains. The resulting synthetic-fixture metric is not evidence of real-world phishing performance. It is combined with transparent offline rules as a URL risk indicator.

Without a URL, the combined risk follows text-model signal. With a URL, a 70% text / 30% URL composition is used. A separate safety rule may raise the indicator for obvious requests for OTPs, passwords or bank details paired with prizes/threats. The output 0-100 is an indicative score, not a calibrated probability.

Why HTTPS does not prove trust

A phishing page may present a valid HTTPS certificate. The scheme is just one feature and must never be treated as an allow-list.

Security / privacy

Private and reserved IPs are rejected for optional external WHOIS queries. The live demo does not browse submitted URLs or log message histories. Public-share Colab links require explicit privacy caution.

06 | Explainability, API and web interface

User-facing explanations

Users see the verdict, detected language, risk indicator, suspicious patterns and URL-feature warnings. These descriptions are pattern-based, human-readable reasons; they are not falsely presented as SHAP scores.

SHAP explanations

The included script successfully computed text-token SHAP contributions to a selected prediction and Tree SHAP contributions from the URL Random Forest. Actual feature-attribution CSV outputs are included. SHAP explanations reflect the limitations of their underlying, provisionally trained models.

Run: python scripts/run_shap.py "You won a prize, verify now at https://bit.ly/claim-bonus"

REST API

The FastAPI endpoint is POST /analyze, accepting JSON with a message and returning verdict, risk indicator, detected language, class scores, extracted URLs and explanations. Validation rejects blank or very long messages. GET /health provides a status response.

Example input and output

Input (Roman Urdu): Apka account band hoga, foran apna OTP bhejein.

Observed demo response: Scam, indicative risk 85/100. The safety-rule floor detected an urgent sensitive-data request, overriding weak model confidence. It is a demo result, not a confirmed classification of a real received message.

User interface

The Gradio UI has a message box, Check Message button, color-coded result, written explanation, URLs and JSON details. A Colab notebook provides a temporary web link, while local launch operates entirely on a user machine.

07 | Tests, failure analysis and limitations

Automated tests

The current build passed 18 automated pytest checks across Unicode normalization, language heuristic, URL parsing, offline safety, data splits, API input validation, and representative trilingual messages. Unit tests do not replace independent evaluation against new real-world scams.

Major failure modes

• Only nine provisional Suspicious cases landed in the current holdout; Suspicious F1 was very weak.

• Urdu-script model generalization is poor and real independently confirmed Urdu phishing texts are scarce.

• Some public source Fraud labels appear questionable, and no individual label was approved by a reviewer.

• Synthetic training examples can inflate performance or create misleading feature patterns.

• Random Forest URL fixture uses generated URLs, producing unrealistically strong test performance.

• Heuristic language detection may confuse Roman Urdu with English and mixed-language messages.

Ethical handling

Collect only consented or appropriately licensed public messages; mask names, phone numbers, OTPs, passwords and tracking links. Never open malicious URLs during collection. Keep private data out of public repositories. Do not claim certainty or present the tool as an alternative to contacting the real bank/provider.

Research conclusion

The implementation demonstrates a complete software architecture for three-language scam risk screening and explanation. It does not yet establish an academically defensible detection rate on independently labeled Pakistani real-world messages.

08 | Reproduction and required next experiments

How to run

Install Python 3.10 or newer. Extract the ZIP, install requirements, then run python run.py; open http://127.0.0.1:7860. For backend docs run uvicorn phishguard.api:app --port 8000 then visit http://127.0.0.1:8000/docs. Run python -m pytest -q for tests.

Requirements not yet independently satisfied

• Independently review a genuinely representative three-language 3,000-5,000 message dataset; do not count synthetic examples as real.

• Create a locked, separate collection of 50-100 genuinely new, redacted, manually verified messages and complete external error analysis.

• Train XLM-RoBERTa on a free/available GPU, then compare with baselines using the same reviewed split.

• Obtain properly labeled and licensed URL datasets to replace synthetic URL fixtures; calibrate risk probabilities.

• Check source licences, publish to the student-owned GitHub account, and adapt the report to the institution's format.

Repository orientation

Key files: `phishguard/preprocessing.py`, `phishguard/urls.py`, `phishguard/inference.py`, `phishguard/api.py`, `phishguard/ui.py`, `scripts/train_text.py`, `scripts/train_urls.py`, `scripts/run_shap.py`, `scripts/train_transformer.py`, `data/`, `reports/`, `tests/`, and `colab/`.

Research and tool references

Original project requirements PDF (user-provided); UCI SMS Spam Collection; Zenodo Roman Urdu candidate dataset; Pak Spam Guard public dataset; Pakistan 2026 SMS research dataset; documentation for scikit-learn, SHAP, Hugging Face Transformers, FastAPI and Gradio. Verify each source license and exact bibliographic details before thesis submission.